A Reproducible BB84 Quantum Key Distribution Simulator with Illustrative Finite-Key Analysis

Lam Thanh Dat
Department of Telecommunications
Faculty of Electrical and Electronics Engineering
Ho Chi Minh City University of Technology (HCMUT), VNU-HCM
Ho Chi Minh City, Vietnam

***Abstract*—Quantum key distribution exploits measurement disturbance to reveal eavesdropping during secret-key establishment. This paper presents a reproducible Python simulator of the Bennett–Brassard 1984 protocol under detection loss, intrinsic bit errors, intercept–resend attacks, heuristic information leakage, and finite data blocks. The implementation separates an asymptotic secret-fraction model from an illustrative finite-size correction and provides validated command-line and graphical interfaces, deterministic parameter sweeps, and automated tests. In a reference run with one hundred thousand emitted signal slots, an ideal link produced zero quantum bit error rate, whereas a full intercept–resend attack produced a rate of 24.79 percent and eliminated the estimated secret-key budget. For an error-correction inefficiency of 1.15, the implemented asymptotic equation gives a critical error rate of 9.88 percent. Finite-size sweeps show that the gap from the asymptotic estimate decreases with increasing block size. The simulator is intended for education, model verification, and sensitivity analysis; it is not a composable security proof or a deployable quantum key distribution system.**

***Index Terms*—BB84, finite-key analysis, Monte Carlo simulation, quantum communication, quantum key distribution, reproducible research.**

I. Introduction

The Bennett–Brassard 1984 (BB84) protocol encodes classical information in nonorthogonal quantum states and uses public basis reconciliation to form a shared sifted key [1]. An eavesdropper who measures unknown states generally disturbs them, allowing the legitimate users to estimate the quantum bit error rate (QBER) and abort when the disturbance is excessive. Security proofs relate the observed bit and phase errors to an extractable secret-key length under explicit protocol assumptions [2].

Educational implementations can reproduce this qualitative behavior, but their numerical output is meaningful only when the channel abstraction, postprocessing budget, and finite-size assumptions are stated precisely. In particular, a Monte Carlo confidence interval quantifies variation across simulated trials; it is not a cryptographic security bound. Likewise, an estimated secret length is not equivalent to executing error correction, verification, authentication, and privacy amplification.

This work standardizes a project-scale BB84 simulator and makes three contributions. First, it separates detection loss, intrinsic bit errors, and intercept–resend disturbance instead of combining them in one ambiguous efficiency parameter. Second, it implements deterministic experiments, machine-readable outputs, input validation, and automated tests. Third, it audits the equations used by the source project and corrects the critical QBER associated with the selected error-correction inefficiency. The contribution is therefore reproducible modeling and software engineering rather than a new QKD security proof.

II. System Model

A. Signal preparation, detection, and sifting

For each of N emitted signal slots, Alice independently samples a bit and one of the Z and X bases. Bob independently selects a measurement basis. A detected event is retained only when the bases agree. Two event detection models are exposed:

$$p_{\mathrm{det}} = \begin{cases} \min(\mu\eta, 1), & \text{linear approximation,} \\ 1 - \exp(-\mu\eta), & \text{Poisson approximation,} \end{cases} \tag{1}$$

where μ is the mean photon number or normalized source intensity and η is the receiver-efficiency factor. The first expression preserves compatibility with the source project. The second is the at-least-one-event probability of the simplified Poisson model. Neither expression includes dark counts or detector dead time.

The parameter F_b denotes intrinsic bit fidelity, so the independent channel-flip probability is $1 - F_b$. Eve attacks each signal independently with probability p_{atk}, measures it in a random BB84 basis, and resends the result. In an otherwise ideal link, Eve selects the wrong basis with probability one half and Bob then obtains a wrong bit with probability one half. The expected attack-induced QBER is consequently

$$\mathbb{E}[Q_{\mathrm{atk}}] = \frac{1}{4} p_{\mathrm{atk}}. \tag{2}$$

B. Asymptotic secret-fraction estimate

For $0 \le x \le 1$, the binary entropy is

$$h_2(x) = -x \log_2 x - (1 - x) \log_2 (1 - x), \tag{3}$$

with the endpoint values defined by continuity. Let Q be the measured QBER, $f_{\mathrm{EC}} \ge 1$ the error-correction inefficiency, and λ a heuristic fractional leakage penalty. The simulator

estimates the asymptotic secret fraction per retained sifted bit as

$$r_\infty = [1 - (1 + f_{\mathrm{EC}})h_2(Q)]^+ (1 - \lambda), \tag{4}$$

where $[x]^+ = \max(x, 0)$. Equation (4) is an information budget; no reconciliation code or privacy-amplification hash is executed. For $f_{\mathrm{EC}} = 1.15$, the zero-key condition $(1 + f_{\mathrm{EC}})h_2(Q_{\mathrm{crit}}) = 1$ gives $Q_{\mathrm{crit}} = 0.09878$, or approximately 9.88%.

C. Illustrative finite-size estimate

For n sifted bits, the finite-size module expands the observed QBER using a normal-quantile margin:

$$\delta = z_{1-\varepsilon_{\mathrm{sec}}/2} \sqrt{\frac{\max\{Q(1-Q), 10^{-12}\}}{n}}, \tag{5}$$

$$Q_{\mathrm{ph}}^U = \min\left(Q + \delta, \frac{1}{2}\right), \tag{6}$$

where $\varepsilon_{\mathrm{sec}}$ is the configured secrecy parameter. The provisional bit budget is

$$\ell = n\big[1 - f_{\mathrm{EC}}h_2(Q) - h_2\big(Q_{\mathrm{ph}}^U\big)\big] - \Delta_{\mathrm{sec}}, \tag{7}$$

$$\Delta_{\mathrm{sec}} = 2\log_2\frac{1}{\varepsilon_{\mathrm{sec}}} + \log_2\frac{2}{\varepsilon_{\mathrm{cor}}}, \tag{8}$$

and the reported length is $\ell_{\mathrm{fin}} = \lfloor (1-\lambda)\max(0, \ell) \rfloor$. Here, this construction is used only for sensitivity analysis. Composable finite-key security requires a protocol-specific treatment with rigorous random-sampling bounds, single-photon parameter estimation, and complete allocation of failure probabilities [3].

III. Implementation and Reproducibility

The software is organized as an installable Python package. Immutable parameter and result records provide explicit interfaces for the asymptotic and finite-size models. Validation rejects probabilities outside the unit interval, negative source intensity, nonpositive block size, and $f_{\mathrm{EC}} < 1$. A command-line interface supports single-run, finite-size, and threshold calculations; a desktop interface calls the same validated model functions.

The reproduction script creates reference JSON, comma-separated tables, and vector and raster figures. Independent deterministic seeds are assigned to each sweep point and repetition. The unit-test suite covers entropy endpoints, detection models, attack behavior, threshold calculation, finite-size trends, reproducibility, invalid inputs, result synchronization, and command-line output. The numerical configurations used in the paper are summarized in Table I.

TABLE I
Configurations for the Reported Sweeps

Parameter	Attack sweep	Finite-size sweep
Emitted slots N	2×10^5	10^4 to 3×10^6
Mean photon number μ	1.0	0.4
Intrinsic bit fidelity F_b	1.0	0.98
Receiver efficiency η	1.0	0.9
Attack probability p_{atk}	0 to 1	0
Repetitions per point	8	5
$\varepsilon_{\mathrm{sec}}, \varepsilon_{\mathrm{cor}}$	—	$10^{-9}, 10^{-9}$

TABLE II
Deterministic Reference Scenarios

Scenario	Q	r_∞	Key bits
Ideal baseline	0.0000	1.0000	50001
Full intercept–resend	0.2479	0.0000	0
Five-percent intrinsic error	0.0513	0.3722	18611
Above-threshold error	0.1361	0.0000	0

IV. Numerical Results

A. Reference scenarios

Table II reports deterministic runs with 10^5 emitted signal slots and seed 2025. The ideal, intercept–resend, and intrinsic-error cases use $\mu = \eta = 1$, $f_{\mathrm{EC}} = 1.15$, no heuristic leakage, and 192 disclosed-bit deductions after QBER evaluation.

The full intercept–resend case gives $Q = 0.2479$, consistent with (2). The five-percent intrinsic-error case retains a positive budget, whereas the above-threshold case is clipped to zero by (4). The calculated threshold of 9.88% corrects the 10.3% value stated in the source report for the same equation and f_{EC}.

B. Attack sweep

As shown in Fig. 1(a), the mean QBER follows $0.25p_{\mathrm{atk}}$ over the tested range. At full attack, the eight-run mean is 0.24979. The estimated secret fraction decreases nonlinearly because QBER enters the entropy function. It remains 0.1728 at $p_{\mathrm{atk}} = 0.3$, where the mean QBER is 0.0751, and becomes zero at $p_{\mathrm{atk}} = 0.4$, where the mean QBER is 0.1003. The transition agrees with the calculated critical QBER.

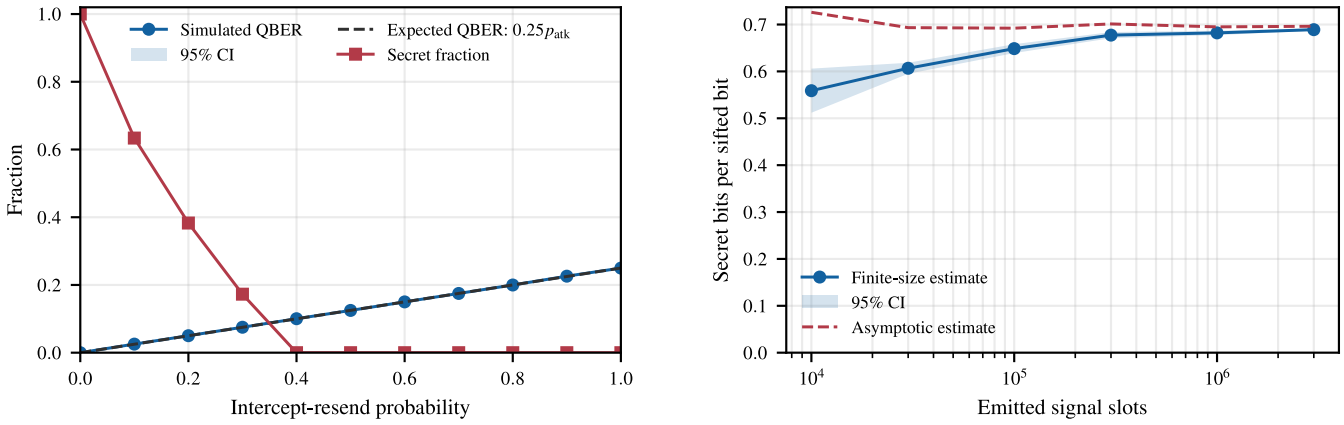
C. Finite-size trend

The finite-size sweep in Fig. 1(b) uses the Poisson detection model and the settings in Table I. At $N = 10^4$, the mean finite-size fraction is 0.5588, compared with an asymptotic estimate of 0.7261. At $N = 3 \times 10^6$, the corresponding values are 0.6890 and 0.6963. The shrinking gap is consistent with the $n^{-1/2}$ statistical margin in (5). These results also show that a universal zero-key cutoff below 10^5 emitted slots is not supported by this parameter set.

V. Discussion and Limitations

The experiments reproduce the principal qualitative behavior expected from the implemented abstraction. Intercept–resend disturbance raises QBER approximately linearly, the entropy-based secret budget collapses after the model-specific threshold, and finite data impose a penalty that decreases as the block grows. Detection loss reduces the number of detected and sifted events; in the present model it does not itself introduce errors.

The numerical outputs must not be interpreted as certified secure-key rates. QBER is evaluated over the entire sifted sequence before a fixed disclosure count is deducted, rather than being estimated from the disclosed sample. The deducted positions are selected through a seed-reproducible random mask for retained-bit bookkeeping, but sampling uncertainty



(a) (b)

Fig. 1. Reproducible Monte Carlo sweeps. (a) QBER and the asymptotic secret fraction versus intercept–resend probability. (b) Illustrative finite-size secret fraction versus emitted signal slots. The shaded bands are 95% normal confidence intervals for the simulated mean; they are neither experimental uncertainty intervals nor cryptographic security bounds.

is not propagated. The simulator accounts for error-correction and privacy-amplification leakage but does not execute either algorithm. It also omits authentication, dark counts, afterpulsing, dead time, timing jitter, source fluctuations, detector-efficiency mismatch, and correlated noise.

The source model does not estimate the yield or error rate of the single-photon component. Its heuristic leakage penalty is not a decoy-state calculation [4], [5]. The code also omits the detector-side protection of measurement-device-independent QKD. The normal approximation in (5) is not a substitute for a tight composable finite-key proof. These restrictions are material: removing them requires a new protocol model, not merely additional simulation trials [6], [7].

VI. Conclusion

A report-scale BB84 program has been converted into a reproducible research repository and documented with explicit mathematical and security boundaries. The simulator validates the expected one-quarter QBER under full intercept–resend, identifies a 9.88% critical QBER for the implemented model with $f_{\text{EC}} = 1.15$, and demonstrates convergence of an illustrative finite-size estimate toward its asymptotic counterpart. Future work should implement decoy-state single-photon bounds, rigorous concentration inequalities, authenticated classical communication, error-correction codes, and universal-hash privacy amplification before physical secure-key-rate claims are considered.

Acknowledgment

The author thanks Prof. Le Tien Thuong for supervision. OpenAI's ChatGPT, including Codex, was used to assist with English-language editing, refactoring of the accompanying Python implementation, preparation of software tests, and initial drafting of the abstract and explanatory prose in Sections III–VI. The author retains responsibility for the technical content and final approval.

References

[1] C. H. Bennett and G. Brassard, "Quantum cryptography: Public key distribution and coin tossing," in *Proc. IEEE Int. Conf. Computers, Systems and Signal Processing*, Bangalore, India, 1984, pp. 175–179.

[2] P. W. Shor and J. Preskill, "Simple proof of security of the BB84 quantum key distribution protocol," *Phys. Rev. Lett.*, vol. 85, no. 2, pp. 441–444, Jul. 2000, doi: 10.1103/PhysRevLett.85.441.

[3] M. Tomamichel, C. C. W. Lim, N. Gisin, and R. Renner, "Tight finite-key analysis for quantum cryptography," *Nat. Commun.*, vol. 3, Art. no. 634, Jan. 2012, doi: 10.1038/ncomms1631.

[4] H.-K. Lo, X. Ma, and K. Chen, "Decoy state quantum key distribution," *Phys. Rev. Lett.*, vol. 94, no. 23, Art. no. 230504, Jun. 2005, doi: 10.1103/PhysRevLett.94.230504.

[5] X. Ma, B. Qi, Y. Zhao, and H.-K. Lo, "Practical decoy state for quantum key distribution," *Phys. Rev. A*, vol. 72, no. 1, Art. no. 012326, Jul. 2005, doi: 10.1103/PhysRevA.72.012326.

[6] V. Scarani, H. Bechmann-Pasquinucci, N. J. Cerf, M. Dušek, N. Lütkenhaus, and M. Peev, "The security of practical quantum key distribution," *Rev. Mod. Phys.*, vol. 81, no. 3, pp. 1301–1350, Sep. 2009, doi: 10.1103/RevModPhys.81.1301.

[7] F. Xu, X. Ma, Q. Zhang, H.-K. Lo, and J.-W. Pan, "Secure quantum key distribution with realistic devices," *Rev. Mod. Phys.*, vol. 92, no. 2, Art. no. 025002, May 2020, doi: 10.1103/RevModPhys.92.025002.